

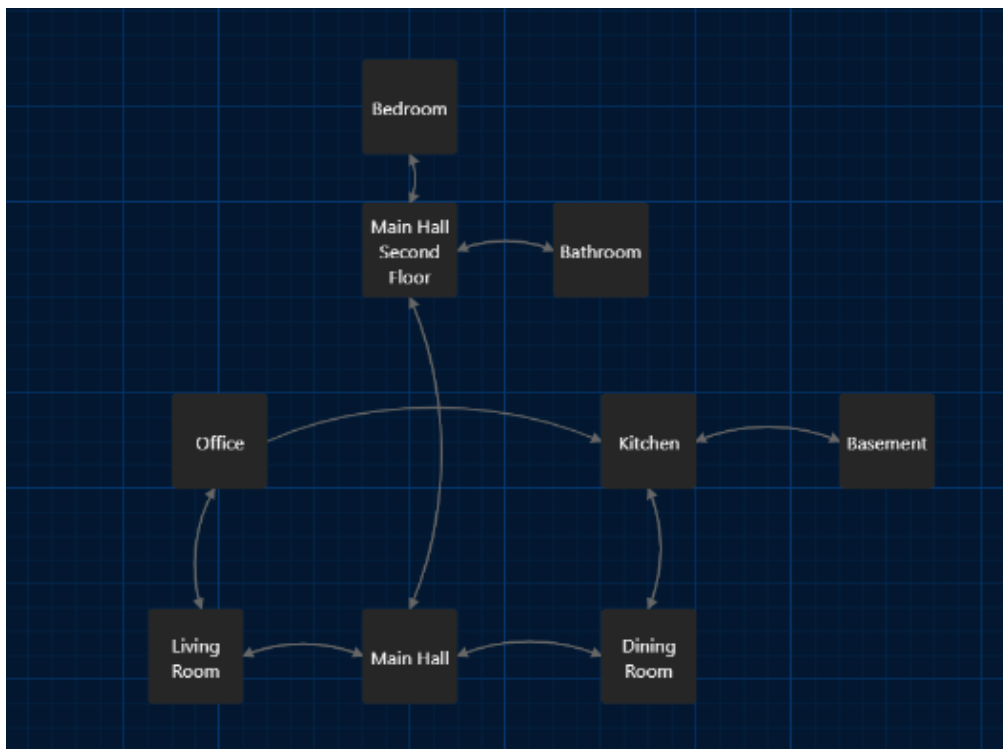
PROJECT TITLE: Thief game
VERSION or DATE: v12, 17 oct 2024
AUTHORS: Jeremy AUGIER
SOURCES: cours java de mr BUREAU

Thème : jeu de discrétion

Résumé du scénario :

Vous êtes un voleur et vous devez vous emparer des objets de plus grande valeur avant de vous faire découvrir.

Plan :



Détail des lieux :

Hall d'entrée - objets
Salon - objets - chat
Bureau - objets cachés - porte à sens unique
Salle à manger - objets
Cuisine - objets
Sous sol - objets
Deuxième étage - rien
Salle de bain - rien
Chambre - porte fermée - objets cachés

Pas de "victoire", il y a un score en fin de partie
Défaite si on fait trop de bruit

Mode d'emploi :

Utiliser "look" et "use" sur tout ce qu'on trouve

-- Exercices --

7.5

print fait avec des System.out.print()
location info ne contient qu'un détail des Rooms

7.6

getExit(final String pExit) --> retourne la Room correspondant à la sortie pExit

7.7

getExitString()
retourne un String énumérant les exits

7.8

on utilise une HashMap<String, Room> pour stocker les Rooms
on utilise désormais vExitMap.get("nomRoom") pour récupérer les objets Room associés aux exits

7.9

keySet() permet de récupérer un Set contenant les clés de la HashMap
On s'en sert pour parcourir les objets de la HashMap 1 à 1

7.10

getExitString crée un String et va parcourir la keySet de la HashMap de Rooms avec :
 for (String vDir : this.exits.keySet())
et va ajouter vDir à la fin du String à chaque itération.

7.11

getLongDescription
renvoie un String contenant this.aDescription et d'autres informations

7.14

look() inutile pour l'instant
on rajoute dans interpretCommand() :

```
else if ( vCommandWord.equals( "look" ) ) {  
    this.look( vCommand );  
}
```

7.15

eat() inutile pour l'instant
(changée en use() dans les dernières versions)

```
else if ( vCommandWord.equals( "eat" ) ) {  
    this.eat( vCommand );  
}
```

7.16

showCommands()

rend visible toutes les commandes possibles

(certaines ont été cachées dans les versions antérieures : test, alea)

7.18

getCommandList() renvoie la String contenant toutes les commandes (voir getExitString en 7.10)

7.18.1

Les améliorations proposées par zuul-better ont été mises en place

7.18.2

StringBuilder est une meilleur manière de faire les exercices 7.10 et 7.18 :

```
public String getCommandList()
{
    StringBuilder vList = new StringBuilder(" ");
    for(String vCommand : this.aValidCommands){
        vList.append(vCommand + " ");
    }
    return vList.toString();
} // showAll()
```

7.18.15

Les Rooms sont maintenant rangées dans une HashMap vRoomList<String, Room>

7.18

Images temporaires mises en place

7.18.8

on ajoute un bouton pour l'action "look" :

```
// add button
```

```
this.aButton = new JButton("look");
```

```
...
```

```
vPanel.add( this.aButton, BorderLayout.EAST );
```

```
...
```

```
@Override public void actionPerformed( final ActionEvent pE )
```

```
{
```

```
    if (pE.getActionCommand() == "look") this.aEngine.interpretCommand( "look");
```

```
    else this.processCommand(); // never suppress this line
```

```
} // actionPerformed(.)
```

7.19.2

création du dossier "Images"

7.20

nouvelle class Item

7.21

la description des items est affichée dans printLocationInfo()

7.22

Les Rooms contiennent maintenant une HashMap pour stocker plusieurs Item

7.22.1

Une HashMap est pratique car on peut se servir de Strings comme de clés (plus faciles à manipuler que des objets Item)

7.22.2

Items intégrés dans le jeu

7.23

go stocke la room précédente dans une Stack<Room>

back récupère la dernière Room de la pile avec .pop() et la désigne comme aCurrentRoom

7.26

Stack déjà fait à 7.23

7.28.1

test permet l'utilisation d'un fichier .txt pour effectuer une série de commandes

On utilise un Scanner pour accéder au document et un "try/catch" au cas où il n'existerait pas

```
try {
    vSc = new Scanner( new File( vFichier + ".txt" ) );
    while ( vSc.hasNextLine() ) {
        String vLine = vSc.nextLine();
        this.interpretCommand( vLine );
    }
}
catch ( final FileNotFoundException pNF ) {
    this.aGui.print("\n");
    this.aGui.println( "test not found" );
    this.aGui.print("\n");
}
```

7.28.2

différents fichiers test sont créés pour tester différents aspects du jeu

7.29

nouvelle classe Player

la CurrentRoom est désormais gérée par Player

les déplacements sont gérés en partie (changement de CurrentRoom) par Player

```
public void goNextRoom(final Room pNextRoom)
{
    if ( this.aStockRooms ) this.aPreviousRooms.push(this.aCurrentRoom);
    else this.aStockRooms = true;

    this.aCurrentRoom = pNextRoom;
}
```

7.30

nouvelle HashMap<Item> pour l'inventaire de Player

nouvelles actions "take", "drop" pour accéder à l'inventaire du joueur

--> utilisent des procédures "pickUpItem" et "dropItem" à l'intérieur de Player pour accéder à l'inventaire

7.31

Inventaire sous forme de HashMap<Item> créée en 7.30

7.31.1

nouvelle classe ItemList

permet de gérer les HashMap<Item> de Room et Player sans duplication de code

toutes les fonctions/procédures utilisant les HashMap renvoient maintenant à des fonctions internes de ItemList.

7.32

Notion de poids max ajoutée

le Player ne peut pas ajouter à son inventaire un objet si il atteint le poids max

7.33

commande inventory permet d'afficher l'inventaire de Player

7.34

un objet (Spotion) permet désormais d'augmenter le poids max

--> procédure "use" remplace "eat"

--> le poids max n'empêche plus de porter trop d'items mais se déplacer au-dessus de poids max fait du bruit

--> trop de bruit cause un Game Over

7.42

La "time limite" est représentée par un compteur de "bruit" qui cause un Game Over une fois plein

7.43

Une porte est désormais à sens unique : Office --> Kitchen
la Stack utilisée par back() est clear() pour éviter le retour en arrière
la Room précédente ne fait pas non plus partie des exits possibles pour la nouvelle Room

7.44

ajout d'un objet (tablette) qui stocke une Room dans un attribut de GameEngine et permet de s'y téléporter plus tard (efface la Stack de back())

7.45

ajout d'une porte fermée et d'une clé

7.46

transporter room ajoutée sous la forme d'une Room qui téléporte Player immédiatement dès qu'il y rentre (dans une Room choisie parmi une list de destinations possibles)

7.46.1

commande alea + String qui stocke un String désignant une destination forcée pour la transporter room
alea toute seule libère la mémoire pour laisser la transporter room fonctionner correctement

7.47.1

paquetages créés

7.48

nouvelle classe Character (renommée NPC car Character est déjà reconnu par Java)
les Character ont tous une CurrentRoom et une ItemList

7.49

ajout d'un NPC qui se déplace à chaque action du Player
